# Identificación de Cantos de Aves del Valle del Cauca con TinyML: Captura con Arduino Nano 33 BLE Sense, Validación por Sesión e Inferencia en el Borde

Sebastián Santana, Juan Camilo Díaz y César Armando Reyes
*Facultad de Ingeniería, Universidad Autónoma de Occidente, Cali, Colombia*
*Inteligencia Artificial en Dispositivos Móviles y Embebidos — Prof. Juan Camilo Giraldo Londoño — 27 de octubre de 2026*

***Resumen*—Se presenta un sistema TinyML que identifica, a partir de su canto, seis aves comunes del Valle del Cauca (garrapatero común, pellar común, pigua, bichofué gritón, azulejo común y lora andina) y rechaza cualquier otro sonido mediante una séptima clase, "ave no registrada". El dataset (578 clips, 7 sesiones) se construyó con grabaciones de xeno-canto, capturas con el micrófono PDM de un Arduino Nano 33 BLE Sense a través de Edge Impulse, grabaciones con un smartphone y recortes de los videos de referencia. Siguiendo la metodología de detección de palabras clave (KWS), se extraen a mano 13 coeficientes MFCC por cuadro (25 ms/10 ms, 40 filtros mel) sobre ventanas de 1 s (1.274 características) y se entrena en NumPy un perceptrón multicapa de 82.055 parámetros. La validación separa los datos por sesión de grabación: el split aleatorio reporta 81,0 % de exactitud, mientras que la validación honesta (leave-one-session-out) da 61,7 %, lo que evidencia una fuga de información de 19,3 puntos. El modelo se exporta a TensorFlow.js y corre en el navegador del celular, que actúa como dispositivo de borde: la app web "Modo Cazador" extrae los MFCC en JavaScript (diferencia < 1e-5 respecto a Python) y muestra un mensaje distinto por clase. En condiciones simuladas de uso en clase, el sistema reconoce el 76 % de las grabaciones de los videos de referencia.**

***Palabras clave*—TinyML, bioacústica, MFCC, Edge Impulse, Arduino Nano 33 BLE Sense, TensorFlow.js, validación por sesión, ODS 15.**

## I. Introducción y objetivo

El monitoreo acústico de aves es una herramienta clave de la ecología: muchas especies se detectan antes por su canto que por su aspecto, y sistemas como BirdNET [3] han mostrado que el aprendizaje profundo puede identificarlas a gran escala. Sin embargo, esos modelos dependen de servidores o de teléfonos de gama alta. El aprendizaje automático embebido (TinyML) [4] propone llevar la inferencia a dispositivos de bajo consumo, cerca del sensor, sin enviar audio a la nube.

**Objetivo.** Desarrollar un clasificador de cantos de aves de bajo costo computacional, entrenado con datos propios capturados con hardware embebido, que funcione en el borde (en el dispositivo que escucha) y que sepa decir "no conozco este sonido" cuando no se trata de una de las aves registradas.

**Relación con los ODS.** El proyecto se enmarca en el ODS 15, *Vida de ecosistemas terrestres* [8], cuya meta 15.5 pide detener la pérdida de biodiversidad. Una herramienta de identificación por canto, liviana y sin conexión, facilita inventarios de aves en campo y actividades de ciencia ciudadana. De forma secundaria aporta al ODS 4 (educación de calidad), porque el modo "Desafío de Aves" enseña a reconocer los cantos.

## II. Descripción del proyecto

El sistema tiene tres etapas (Fig. 1): (1) adquisición de un dataset etiquetado por sesión, (2) entrenamiento y validación en Google Colab con NumPy y (3) inferencia en el borde dentro de una app web publicada en GitHub Pages. La app tiene dos modos: el *Desafío de Aves*, un quiz donde suena el canto de referencia y el usuario elige el ave, y el *Identificador IA*, que graba 7 s con el micrófono, calcula los MFCC en el navegador y ejecuta el modelo.

Las clases son las seis aves y la clase de rechazo **ave-no-registrada** (música, ruido de calle, voces y aves que no están en la lista), que hace las veces de la clase "ruido" exigida. Cada clase produce una salida observable distinta en la pantalla (nombre, ícono, porcentaje y barras de probabilidad); la clase de rechazo muestra "Ave no registrada" con las barras en cero.

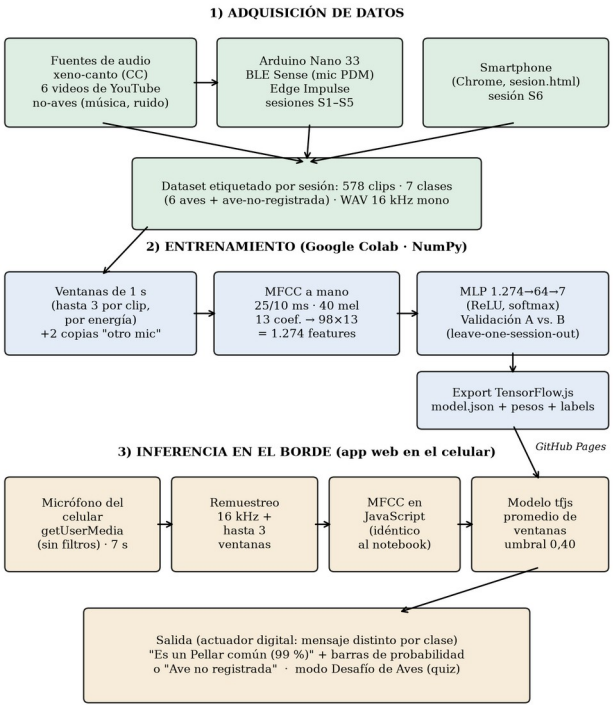



Fig. 1. Diagrama de bloques del sistema: adquisición (fuentes, Nano 33 BLE Sense con Edge Impulse y smartphone), entrenamiento en Colab y exportación a TensorFlow.js, e inferencia en el borde dentro del navegador del celular.

## III. Hardware y software utilizados

TABLA I
COMPONENTES DEL SISTEMA

| Elemento | Uso y características |
|---|---|
| Arduino Nano 33 BLE Sense | Captura de audio para el dataset (S1–S5). nRF52840 Cortex-M4F a 64 MHz, 1 MB Flash, 256 KB RAM, micrófono PDM MP34DT05 [10]. Firmware de Edge Impulse, 16 kHz. |
| Smartphone Android (Chrome) | Dispositivo de borde: ejecuta la app, captura el audio e infiere localmente. También grabó la sesión S6 con app/sesion.html. |
| PC con parlantes (Pop!_OS) | Reproduce los cantos durante la captura y la demostración; ejecuta los scripts de grabación automática. |
| Edge Impulse | Adquisición y etiquetado de las muestras del Nano (proyecto 1150053), exportadas en WAV [6]. |
| Google Colab + NumPy | MFCC, entrenamiento del MLP, validación A vs. B y exportación del modelo (sin TensorFlow). |
| TensorFlow.js 4.20 | Inferencia del modelo en el navegador [7]. |

## IV. Recopilación de datos

El principio rector fue etiquetar cada clip con su **sesión de grabación** (sufijo _S<n>_ en el nombre del archivo), porque los clips de una misma toma comparten micrófono, distancia y ambiente. Así se puede medir si el modelo generaliza a una toma nueva. La Tabla II resume las fuentes.

TABLA II
COMPOSICIÓN DEL DATASET (578 CLIPS, 16 KHZ, MONO)

| Fuente | Sesión | Clips | Clases |
|---|---|---|---|
| xeno-canto (CC) [5] | S1–S4 | 81 | 6 aves |
| Nano 33 BLE Sense vía Edge Impulse | S1–S3 | 91 | 6 aves |
| Música, ruido y otras aves (YouTube) | S1–S3 | 93 | no registrada |
| Nano 33 BLE Sense (aves y no-aves) | S4–S5 | 129 | 7 clases |
| Smartphone con app/sesion.html | S6 | 100 | 7 clases |
| Recortes de los 6 videos de referencia | S7 | 84 | 6 aves |
| Total | 7 | 578 | 7 clases |

**Captura con el Nano.** Un script (grabar_auto.py) reproducía cada canto por los parlantes del PC mientras el Nano, conectado a Edge Impulse, grababa 5 s con la etiqueta <ave>-S<n>. Cada sesión usó un volumen distinto (100 %, 75 % y 55 %; luego 90 % y 65 %) para que el modelo no dependiera de la intensidad. En S4–S5 también se grabó con el Nano la clase de rechazo, normalizando el volumen con loudnorm.

**Captura con el celular.** Para que el modelo conociera el micrófono del dispositivo final se creó app/sesion.html, que graba varios minutos con la misma configuración de micrófono de la app. Un script genera una pista de 10,8 min con 111 clips y señales de sincronía, y otro corta la grabación en un archivo por clip alineando la envolvente de 20 s del original con la grabación (recuperó 100 de 111 clips pese a que la toma empezó tarde y perdió 9 s).

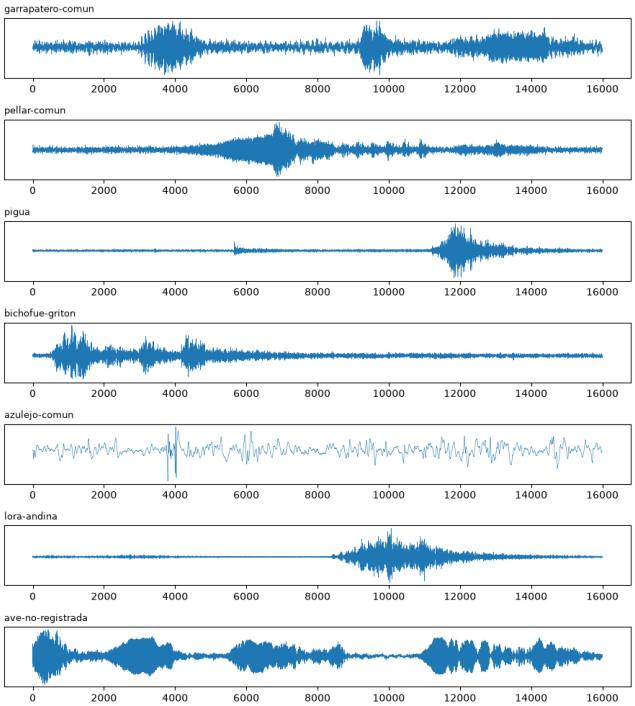


Fig. 2. Forma de onda de un clip por clase (análisis exploratorio del notebook).

## V. Extracción de características y modelo

Se replicó a mano el front-end del ejercicio KWS [2] para que el mismo código exista en Python y en JavaScript: preénfasis 0,97; ventanas de Hann de 25 ms con salto de 10 ms; FFT de 512 puntos; banco de 40 filtros mel entre 50 y 8.000 Hz; logaritmo y DCT para obtener 13 coeficientes [1]. Sobre 1 s de audio se obtienen 98×13 = 1.274 características (Fig. 3). El coeficiente c0, que mide la energía, se anula en la normalización porque depende de la distancia al micrófono y no del ave.

Como el canto no ocupa todo el clip, se toman hasta tres ventanas de 1 s sin solaparse, en los bloques de mayor energía (que tengan al menos el 50 % de la energía del más fuerte); la app promedia las probabilidades de esas mismas ventanas. Para hacer el modelo robusto al micrófono se añadieron dos copias de cada clip con un "canal" aleatorio (pasa-banda, inclinación espectral, resonancia de ±8 dB y ruido); estas copias solo se usan para entrenar, nunca para medir.

El clasificador es un perceptrón multicapa 1.274→64 (ReLU)→7 (softmax) con 82.055 parámetros (328 KB en float32), entrenado con descenso de gradiente por minilotes (lote 16, tasa 0,01, 60 épocas en validación y 80 en el modelo final) y pesos por clase para compensar el desbalance. Se implementó en NumPy y se exportó escribiendo directamente el formato *layers-model* de TensorFlow.js, lo que evita conflictos de versiones en Colab.

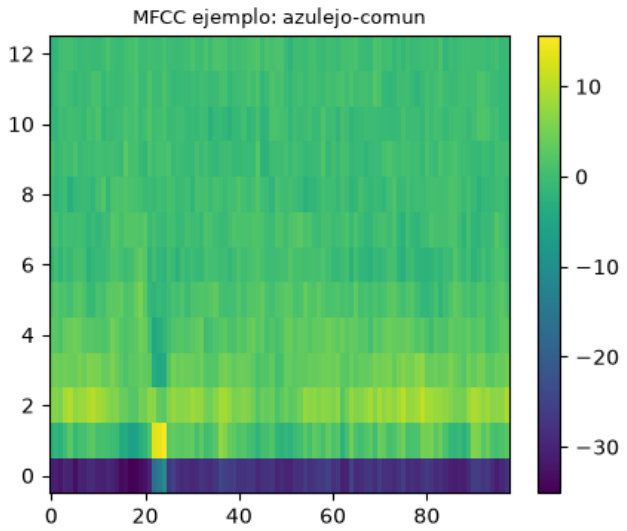



Fig. 3. Matriz MFCC (13 coeficientes × 98 cuadros) de un clip de ejemplo.

## VI. Despliegue en el borde

La app (HTML, CSS y JavaScript sin dependencias pesadas) carga el modelo TensorFlow.js desde GitHub Pages. Al grabar, pide el micrófono con echoCancellation, noiseSuppression y autoGainControl desactivados, porque esos filtros del celular borran parte del canto. El audio se graba 7 s a 128 kbps, se remuestrea a 16 kHz y pasa por las mismas funciones del notebook: ventanas por energía, MFCC y normalización con la media y la desviación del entrenamiento. Se verificó que los MFCC en JavaScript y en Python difieren menos de 1e-5 y que la salida de TensorFlow.js coincide con la de NumPy.

**Regla de decisión.** El sistema responde "Ave no registrada" si gana la clase de rechazo, si la mejor ave no supera un umbral de 0,40 o si el pico de la señal es menor que 0,02 (silencio). En otro caso muestra "Es un <ave> (<p> %)" con su ícono, más las barras de las seis aves. El modo diagnóstico (?debug=1) muestra las probabilidades crudas y permite descargar el audio grabado.

## VII. Resultados

**Experimento A vs. B.** Con un split aleatorio por clip (A), clips de la misma sesión quedan en entrenamiento y prueba, y la exactitud es de 81,0 %. Dejando fuera una sesión completa cada vez (B, leave-one-session-out), la exactitud promedio cae a 61,7 % (Tabla III). Esa diferencia de 19,3 puntos es la fuga de sesión: el modelo de A "reconoce la toma" y no solo el ave. La métrica que se reporta como honesta es la de B.

TABLA III
Exactitud por sesión excluida (experimento B)

| Sesión de prueba | Clips | Exactitud |
|---|---|---|
| S1 (xeno-canto, Nano, no-aves) | 84 | 51,2 % |
| S2 (xeno-canto, Nano, no-aves) | 86 | 67,4 % |
| S3 (xeno-canto, Nano, no-aves) | 79 | 64,6 % |
| S4 (Nano, xeno-canto) | 82 | 82,9 % |
| S5 (Nano) | 63 | 71,4 % |
| S6 (smartphone) | 100 | 86,0 % |
| S7 (videos de referencia) | 84 | 8,3 % |
| Promedio B / split aleatorio A | — | 61,7 % / 81,0 % |

La sesión S7 merece atención: un modelo que nunca escuchó los videos de referencia solo acierta el 8,3 % de sus recortes, porque esas grabaciones (otro micrófono, otra edición y otro individuo) difieren mucho de las de xeno-canto. Por eso los videos se incluyeron en el modelo final. La Fig. 4 muestra la matriz de confusión acumulada de B: la lora andina (70 %) y la pigua (71 %) son las aves mejor reconocidas, mientras que el garrapatero (35 %) y el azulejo (43 %) se confunden con frecuencia con la clase de rechazo.

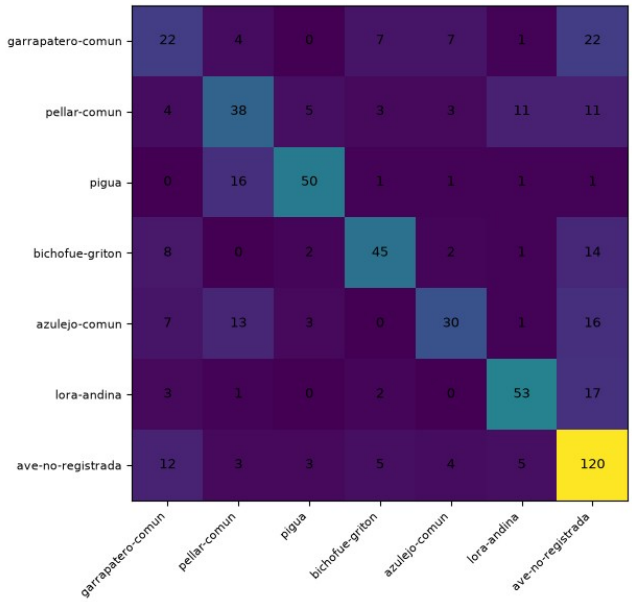



Fig. 4. Matriz de confusión del experimento B (filas: clase real; columnas: predicción), acumulada sobre las 7 sesiones excluidas.

TABLA IV
Efecto del umbral de decisión (experimento B)

| Umbral | No-aves rechazadas | Aves aceptadas y bien nombradas |
|---|---|---|
| 0,40 | 79,6 % | 55,4 % |
| 0,50 | 82,9 % | 53,3 % |
| 0,60 | 88,8 % | 51,2 % |
| 0,70 | 90,1 % | 47,9 % |
| 0,80 | 92,1 % | 44,8 % |

Se eligió el umbral 0,40 porque prioriza nombrar bien las aves y aun así rechaza 4 de cada 5 sonidos ajenos.

**Pruebas de la implementación final.** (1) La app real se ejecutó en Chrome con clips grabados por el smartphone como entrada de micrófono: reconoció correctamente las 9 aves probadas y rechazó los 2 sonidos ajenos. (2) Para imitar la demostración se generaron 72 grabaciones de 7 s de los videos de referencia, con inicio aleatorio, filtros de micrófono distintos a los del entrenamiento, eco, ruido rosa y compresión AAC u Opus. El modelo publicado acertó 55/72 (76 %): pellar común y lora andina 12/12, azulejo 11/12, garrapatero 9/12, bichofué 7/12 y pigua 4/12. Casi todos los errores fueron rechazos ("no

registrada") y no confusiones entre aves, que es el error menos grave.

## VIII. Problemas encontrados y soluciones

- **Fuga de sesión:** el split aleatorio daba resultados inflados. Solución: nombres con _S<n>_ y validación leave-one-session-out (A vs. B).
- **Atajo en la clase de rechazo:** las no-aves venían de YouTube (audio limpio) y las aves del Nano, así que el modelo podía separar "micrófono" en vez de "canto". Solución: grabar también las no-aves con el Nano (S4–S5).
- **Filtros del navegador:** la supresión de ruido y el control de ganancia del celular borraban el canto. Solución: desactivarlos en getUserMedia.
- **Filtro de ruido propio:** un filtro por envolvente y tasa de cruces por cero rechazaba 31 de 91 aves reales. Solución: eliminarlo y dejar la decisión a la clase de rechazo y al umbral.
- **Cambio de micrófono:** el modelo entrenado con el Nano fallaba con otros micrófonos. Solución: aumento de datos con canal simulado y la sesión S6 grabada con el propio celular.
- **Zips que se pisaban:** dos zips tenían archivos con el mismo nombre y al descomprimirlos en la misma carpeta se perdían unos 65 clips. Solución: cada zip en su propia carpeta.
- **Videos de referencia fuera del dataset:** la app se probaba con los videos de YouTube, que el modelo nunca había escuchado (8,3 %). Solución: sesión S7 con recortes de esos videos.
- **Micrófono del PC:** el micrófono interno del portátil tenía una relación señal/ruido ≤ 3, insuficiente. Solución: usar la app en el celular y reproducir los cantos desde el PC.
- **Sincronización de la sesión del celular:** la grabación empezó 2 s tarde y perdió 9 s. Solución: alineación por envolvente con tramos de 20 s y desfases confirmados por varios tramos consecutivos.
- **Compresión del navegador:** a la tasa por defecto (~32 kbps) la confianza del modelo bajaba. Solución: grabar a 128 kbps.
- **Reproducibilidad en Colab:** había que subir los zips a mano. Solución: el notebook descarga los 6 zips del repositorio; desde una carpeta vacía reproduce el modelo publicado bit a bit.

## IX. Limitaciones y trabajo futuro

Los cantos provienen de grabaciones reproducidas por parlante, no de aves en vivo, y varias sesiones repiten audios de origen. Las especies con videos de referencia muy cortos (pigua 5 s y bichofué 6 s) tienen pocos ejemplos y son las que más se rechazan. La inferencia final corre en el navegador del celular; el Nano 33 BLE Sense se usó para capturar datos. Como trabajo futuro se propone exportar el mismo MLP a TensorFlow Lite for Microcontrollers (cuantizado a int8, ~82 KB) para inferir directamente en el Nano con batería y enviar el resultado por BLE, y ampliar el dataset con grabaciones de campo.

## X. Conclusiones

Se construyó un identificador de cantos de aves con un pipeline transparente (MFCC a mano y un MLP pequeño) que se ejecuta igual en Python y en el navegador. El hallazgo más importante es metodológico: la exactitud depende sobre todo de que el dataset incluya el micrófono y las fuentes con las que se usará el sistema. La validación por sesión mostró que el split aleatorio sobreestima el desempeño en 19 puntos, y las sesiones grabadas con el celular y con los videos de referencia fueron las que llevaron el sistema a un desempeño útil en condiciones de uso (76 % en la simulación de la demostración).

**Código y dataset:**
github.com/santa250613/miniproyecto2_tinyml
**App:** santa250613.github.io/miniproyecto2_tinyml
**Notebook (Colab):** notebooks/02_entrenamiento_aves.ipynb
**Edge Impulse (público):**
studio.edgeimpulse.com/public/1150053/latest

## Referencias


[1] S. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," IEEE Trans. Acoust., Speech, Signal Process., vol. 28, no. 4, pp. 357–366, 1980.

[2] P. Warden, "Speech Commands: A dataset for limited-vocabulary speech recognition," arXiv:1804.03209, 2018.

[3] S. Kahl, C. M. Wood, M. Eibl, and H. Klinck, "BirdNET: A deep learning solution for avian diversity monitoring," Ecological Informatics, vol. 61, art. 101236, 2021.

[4] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. Sebastopol, CA, USA: O'Reilly, 2019.

[5] Xeno-canto Foundation, "xeno-canto: Sharing wildlife sounds from around the world." [Online]. Available: https://xeno-canto.org

[6] Edge Impulse, "Edge Impulse documentation." [Online]. Available: https://docs.edgeimpulse.com

[7] D. Smilkov et al., "TensorFlow.js: Machine learning for the web and beyond," in Proc. SysML Conf., 2019.

[8] Naciones Unidas, "Objetivo 15: Vida de ecosistemas terrestres." [Online]. Available: https://www.un.org/sustainabledevelopment/es/biodiversity/

[9] D. Stowell, M. D. Wood, H. Pamuła, Y. Stylianou, and H. Glotin, "Automatic acoustic detection of birds through deep learning: The first Bird Audio Detection challenge," Methods Ecol. Evol., vol. 10, no. 3, pp. 368–380, 2019.

[10] Arduino, "Arduino Nano 33 BLE Sense — Product reference manual (ABX00031)." [Online]. Available: https://docs.arduino.cc

[11] Miniproyecto 1 — Bartender TinyML (metodología KWS de referencia). [Online]. Available: https://github.com/caesar-dat-com/miniproyecto1-tinyml